实验一　性能评价与仿真方法学

4 学时　|　贯穿负载：qnn（int8 量化神经网络推理）

本次实验要你做出的决策

> **后面六次实验，你打算用哪个 CPU 模型？为什么？你测出来的数，误差有多大，这个误差会不会让你选错方案？**

这不是一道有标准答案的题。gem5 提供四个精度和速度差了几十倍的 CPU 模型，选快的省时间但可能得出错误结论，选准的结论可靠但一次扫描要跑一小时。本次实验结束时，你要拿出**数据**来支持你的选择。

一、为什么体系结构研究离不开模拟器

设计一款处理器，流片一次的掩模费用是几百万到上亿人民币。你不可能" 先造出来再看快不快"。所以在 RTL 之前，必须有办法回答" 如果我把 L1 加倍，性能会涨多少"。

评价手段大致三档：

手段	速度	精度	什么时候用
解析模型（如 Roofline）	秒级	量级正确	早期筛掉明显不可行的方案
周期级模拟器（gem5）	慢 10^4~10^6 倍	5%~20% 误差	**设计空间探索，本课程用它**
FPGA 原型 / 流片	接近实时	精确	方案定稿后验证

"慢 10^4 倍" 是什么概念：真机 1 秒的程序，gem5 要跑约 3 小时。所以**怎么把测量成本压下来而又不失真**，本身就是一门手艺——这就是" 方法学"。

gem5 是什么

gem5 是一个开源的**周期级全系统模拟器**。它逐条解释目标 ISA（我们用 RISC-V）的指令，同时用软件模拟出流水线、Cache、总线、DRAM 控制器的时序行为，最后告诉你 "这段程序在这个假想的芯片上要跑多少个周期"。

本课程用它的 **SE 模式**（System-call Emulation，系统调用模拟）：不启动操作系统，程序的系统调用由 gem5 直接代办。好处是启动快、配置简单；代价是测不了 OS 相关的行为——对我们研究 Cache、流水线、加速器完全够用。

二、性能到底怎么量

2.1 唯一可靠的性能定义

《计算机组成原理》里的公式仍然成立，而且是本课程一切讨论的地基：

$$\text{CPU 时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期}$$

三个因子分别被什么决定：

因子	主要由谁决定	本课程哪次实验动它
指令数	ISA、编译器、算法	**第 5、6 次**（自定义指令直接砍指令数）
CPI	微体系结构（流水线、Cache、分支预测）	第 2、3、4 次
时钟周期	工艺、流水级划分	第 7 次（作为约束条件）

2.2 为什么不能只看 IPC

IPC（每周期指令数）= 1/CPI，是模拟器最爱报的指标，但它**不是性能**。

> 反例：你把一段循环用一条自定义指令替代，指令数从 100 万降到 10 万，但这条新指令很复杂，IPC 从 5.0 掉到 1.0。IPC 看起来” 变差了 5 倍”，实际执行时间从 20 万周期降到 10 万周期，**快了一倍**。

规矩：比较两个方案时，永远报” 周期数” 或” 执行时间”，IPC 只用来解释原因。只有在指令数完全相同时（比如只改 Cache 配置），IPC 才可以直接当性能比。第 5 次实验你会亲眼看到这个陷阱。

2.3 加速比要用几何平均

多个基准程序的加速比取平均时，必须用**几何平均**而不是算术平均：

$$\text{几何平均加速比} = \sqrt[n]{\prod_{i=1}^{n} S_i}$$

原因很简单：算术平均对” 谁当基准” 敏感。A 相对 B 快 4 倍、B 相对 A 快 0.25 倍，算术平均下 $(4+0.25)/2 = 2.125$，两个方向算出来不互为倒数，是自相矛盾的。几何平均 $\sqrt{(4\times0.25)} = 1$，才自洽。

三、ROI：你到底在测什么

3.1 一个程序不是只有你写的那部分

一个静态链接的 C 程序在 `main()` 之前要做很多事：设置栈、初始化 libc、初始化 malloc 的堆、处理 `argv/envp`、跑 `.init_array` 里的构造函数。这些代码**和你要研究的算法毫无关系**，但它会被计入统计。

本次实验你会亲手量出来：`qnn 8` 的启动段只占一小部分指令，却吃掉了**远超比例**的周期。如果你不划定测量区域，你测的是 glibc 的启动性能。

ROI（Region of Interest，感兴趣区域）就是你在源码里明确标出的“从这里开始计数，到这里停止”的一段。

3.2 gem5 伪指令是怎么工作的

打开 `bench/include/bench.h`，你会看到：

```
#define M5OP_RAW(encoding)                          \
  do {                                              \
    register uint64_t _a0 __asm__("a0") = 0;        \
    register uint64_t _a1 __asm__("a1") = 0;        \
    __asm__ __volatile__(".long " #encoding         \
                         : "+r"(_a0)                \
                         : "r"(_a1)                 \
                         : "memory");               \
  } while (0)

#define ROI_BEGIN() M5OP_RAW(0x8400007b)   /* 转储并重置统计 */
#define ROI_END()   M5OP_RAW(0x8400007b)   /* 转储并重置统计 */
```

`.long 0x8400007b` 的意思是”在这里放一个 32 位的立即数，当作一条指令”。拆开这个编码看：

```
0x8400007b = 1000010 0000000000 00000 000 00000 1111011
             └─func─┘                      └opcode┘
              = 0x42                        = 0x7b
```

- 低 7 位 `0x7b` 是 RISC-V 规范里预留给厂商的 **custom-3** 操作码空间；
- 高 7 位（bit 31:25）是 gem5 定义的功能号，`0x40` = 重置统计，`0x42` = **转储并重置**（我们两处都用 `0x42`，理由见下一节）。

在**真实的 RISC-V 处理器**上，这条指令会触发” 非法指令” 异常。在 **gem5** 里，它被解码成一个” 伪指令”，触发模拟器内部的动作而不是计算。

> **这就是软硬件协同的最小案例：**我们在 ISA 的预留空间里，自己规定了一条指令的含义，让软件能指挥硬件（这里是模拟器）做一件新事情。第 5 次实验你会用完全相同的手法，在 **custom-0**（操作码 `0x0b`）空间里定义一条真正会算数的新指令。现在先记住这个套路。

3.3 stats.txt 里为什么有三段

两个标记都用” 转储并重置”，程序退出时 gem5 又自动转储一次，于是 `stats.txt` 被切成三段，**每一段都是一段互不重叠的代码：**

```
第 0 段 = 程序开始 → ROI_BEGIN    → 启动段（libc 初始化等）
第 1 段 = ROI_BEGIN → ROI_END     → 【重要】 内核，这才是我们要的
第 2 段 = ROI_END → 程序退出      → 收尾段
```

`framework/stats.py` **默认取第 1 段**。

> **为什么 `ROI_BEGIN` 也要转储？**只用” 重置”（`0x40`）也能把启动段排除掉，但那样启动段的数据就永远消失了，你没法回答” 它到底占了多少”。改成” 转储并重置”，启动开销自己就显形了，**一次运行、同一个二进制就能量化它**。
>
> 另一种想法是” 编译一个不带标记的版本，用全程序减去 ROI”。这个做法**有陷阱：**两个二进制的代码布局不一样，Cache 冲突模式随之改变，相减会引入伪差异。任务 2 会让你亲自验证这一点。

四、gem5 的四个 CPU 模型

这是本次实验的核心知识点。四个模型建模的细致程度差别巨大：

AtomicSimpleCPU（`--cpu atomic`）

一次执行一条指令，访存**瞬间完成**（返回一个估算的延迟值，但不真的走总线）。不建模流水线、不建模乱序、不建模访存排队。

- **用途：**功能验证、快进到感兴趣的位置、生成 checkpoint
- **不能用来：**得出任何性能结论

TimingSimpleCPU（`--cpu timing`）

仍然一次一条指令，但访存**真的走 Cache 和总线**，会因为失效而阻塞。

- **用途**：研究纯访存行为，且不关心计算与访存的重叠
- **陷阱**：因为完全不重叠，它会**极度高估访存延迟的影响**。本次实验你会看到它给出错误的设计结论。

MinorCPU（`--cpu minor`）

一条顺序（in-order）流水线，建模到流水级，有取指队列、译码、执行、写回。类似 ARM Cortex-A7 这类顺序核。

- **用途**：目标是低功耗顺序核时，这是合适的模型
- **陷阱**：顺序核在一次访存失效后就整体停住，所以它对”多个失效同时进行”完全不敏感

O3CPU（`--cpu o3`）

乱序超标量：取指 → 译码 → 重命名 → 分发 → 乱序发射 → 执行 → 写回 → 顺序提交，带 ROB、发射队列、Load/Store 队列、物理寄存器堆、分支预测器。类似 ARM Cortex-A76 / Intel 酷睿这类高性能核。

- **用途：本课程后续实验的默认模型**（但要你自己用数据论证）
- **代价**：主机仿真速率只有 atomic 的 1/4

一张速查表

	流水线	乱序	分支预测	访存重叠	相对仿真速度
atomic	无	无	无	无	最快
timing	无	无	无	无	快
minor	有	无	有	有限	慢
o3	有	有	有	有	慢

五、误差从哪来，什么样的误差可以容忍

真实处理器 vs gem5 的误差通常在 **10%~30%**。这个数字听起来很吓人，但对设计探索来说往往可以接受——**因为我们关心的是排序，不是绝对值**。

误差的四个来源：

1. **模型误差**——模拟器不建模某些机制（如 TimingSimpleCPU 没有流水线重叠）
2. **参数误差**——Cache 延迟、DRAM 时序这些参数配得对不对
3. **工作负载误差**——8 个样本能代表 10000 个样本吗
4. **测量误差**——ROI 划错了、统计块取错了

保真度的两个层次

层次	要求	谁需要它
绝对保真	IPC 数值接近真机	性能预测、产品规格承诺
相对保真（排序保真）	方案 A 比 B 好，模型也说 A 比 B 好	**设计空间探索——我们的场景**

一个模型只要保住排序，即使绝对值差 3 倍也是有用的；反过来，绝对值再准，只要排序错了就是有害的——它会让你选错方案。

本次实验的任务 4 就是拿这条标准去考四个模型。

六、实验环境

```
cd ~/cssys                          # 或教师指定的目录
# 仓库里没有 gem5.opt，也没有 bench/bin/riscv/qnn，先编译：
cd gem5 && scons build/RISCV/gem5.opt -j8 && cd ..          # 第一次要几十分钟
cd bench && make && make noroi && make noalign && cd ..
export GEM5_BIN=$PWD/gem5/build/RISCV/gem5.opt
ls "$GEM5_BIN" bench/bin/riscv/qnn
```

单点运行的命令长这样：

```
$GEM5_BIN --outdir=out/lab1 framework/cssys_se.py \
    --cmd bench/bin/riscv/qnn --options "8" --cpu o3
```

- `--outdir` 指定结果目录，`stats.txt` 会写在里面
- `--cmd` 是被测的 RISC-V 可执行文件
- `--options` 是传给它的命令行参数（要用引号包住）

看结果：

```
python3 framework/stats.py out/lab1/stats.txt      # 默认第 1 段，即 ROI
python3 framework/stats.py out/lab1/stats.txt 0    # 第 0 段，启动段
python3 framework/stats.py out/lab1/stats.txt 2    # 第 2 段，收尾段
```

扫描多个配置（后面每次实验都要用）：

```
python3 framework/sweep.py --name 我的扫描 \
    --bench qnn --bench-args 8 \
    --vary cpu=atomic,timing,minor,o3 --jobs 4
python3 framework/report.py results/我的扫描.csv
```

`--jobs N` 是并行度。gem5 本身是单线程的，机房 PC 有几个核就开几个。

> 【注意】**防抄袭**：所有基准程序都接受一个 seed 参数，请一律用你自己的学号后四位：`--bench-args "8 1234"`。不同 seed 下权重数据不同，Cache 冲突模式和曲线拐点会有差异，抄别人的数据对不上你自己的曲线。

6.1 你的噪声地板有多高

gem5 是**完全确定性**的：同样的二进制、同样的配置，跑一百遍结果完全一样。但只要有任何一点不同，结果就会有微小的抖动。

一个真实的例子：只是把 `--cmd` 从相对路径改成绝对路径（多了 21 个字符），ROI 的周期数就从 1,099,683 变成 1,099,658，**差了 25 个周期**。原因是 SE 模式下命令行字符串存在初始栈上，字符串变长会让整个栈平移，栈上数据落到的 Cache 组随之改变。

> 这个现象有正式的文献出处：Mytkowicz 等人的 *"Producing Wrong Data Without Doing Anything Obviously Wrong!"*（ASPLOS 2009）。他们发现改一个环境变量的长度就能让测量结果变化好几个百分点。

所以本课程的噪声地板约为 0.01%。这条线很重要：**低于它的差异，你不能解释，也不许解释**。本课程所有实验里，**小于 0.1% 的性能差异一律当作" 没有差异"**。

写报告时，凡是你要声称"A 比 B 好"，先问自己：这个差异是噪声地板的几倍？

七、动手任务

任务 1　跑通并读懂（20 分钟）

用 `--cpu o3` 跑 `qnn 8 <` 你的学号后四位 `>`，然后：

1. 打开 `out/.../stats.txt`，找到并记录：`simInsts`、`system.cpu.numCycles`、`system.cpu.ipc`、`hostSeconds`
2. 数一数文件里有几段 `Begin Simulation Statistics`，说明每一段对应源码的哪一部分
3. 用 `framework/stats.py` 依次打印第 0、1、2 段，对照你手工找的数

要回答：第 2 段的指令数不到一万，IPC 却只有 0.4 左右，为什么这么低？（提示：那段代码在干什么？它有循环吗？）

任务 2　ROI 到底省了什么（30 分钟）

（a）三段各占多少

用任务 1 的那次运行就够了，把三段填进表：

段	指令数	占全程序	周期数	占全程序	IPC
第 0 段　启动					
第 1 段　ROI					
第 2 段　收尾					
合计		100%		100%	

要回答：1. 启动段占全程序指令数的百分之几？占周期数的百分之几？**两个比例为什么差这么多？**用” 周期数 = 指令数 ÷ IPC” 解释。2. 如果拿” 全程序 IPC”（＝合计指令 ÷ 合计周期）当作 `qnn` 的性能指标，相对真实的内核 IPC 会低估多少倍？

（b）加大负载能不能替代 ROI

把样本数改成 64 再跑一次，比较两次的第 0 段：

样本数	启动段指令数	启动段周期数	启动占全程序指令	启动占全程序周期
8				
64				

要回答：启动段的**绝对**开销变了吗？占比呢？由此说明：不划 ROI 时，“把负载加大” 能不能替代 ROI？代价是什么？

（c）“两次测量相减” 靠不靠得住【重点】

`qnn_noroi` 是同一份源码加 `-DNO_M5OPS` 编译的，没有 ROI 标记，所以它的 `stats.txt` 只有一段（整个程序）。

一个很自然的想法是：**启动段 = 全程序 (`qnn_noroi`) − ROI(`qnn`) − 收尾 (`qnn`)**。跑一下 `qnn_noroi 8`，用这个公式算一遍，跟你在 (a) 里直接读出来的启动段比较。

现在把同样的事情在另一对二进制上再做一遍。`qnn_noalign` / `qnn_noalign_noroi` 与上面两个**源码完全相同、优化级别完全相同**，唯一的区别是编译时**没有加对齐参数**（`-falign-functions=64` 等）：

```
for b in qnn qnn_noroi qnn_noalign qnn_noalign_noroi; do
  echo -n "$b  "
  $GEM5_BIN --outdir=out/c_$b framework/cssys_se.py \
      --cmd bench/bin/riscv/$b --options "8" --cpu o3 2>&1 | tail -1
done
```

填表（“全程序周期”＝该二进制所有统计段的周期数之和）：

二进制	有 ROI 标记	有对齐参数	全程序周期数
qnn	有	有	
qnn_noroi	无	有	
qnn_noalign	有	无	
qnn_noalign_noroi	无	无	

要回答：1. 在**对齐**的那一对里，相减法的结果和直接测量差多少？2. 在**没对齐**的那一对里，两个二进制的全程序周期数差几倍？它们的**指令数**差多少？3. 用 `riscv64-linux-gnu-nm bench/bin/riscv/<名字> | grep dot_i8` 查出四个二进制里热函数 `dot_i8` 的地址，算出 `地址 mod 64`。把这一列和上表的周期数放在一起看，你发现了什么？

> 【注意】这里有个坑。你大概会想写”加了对齐参数就快”。**先别急**——仔细看 `qnn_noalign` 这一行，它**没加**对齐参数，它慢了吗？把四行的 `mod 64` 和周期数一一对应着再看一遍，然后重新组织你的结论。4. 已知 O3 核的取指单元**每周期只能从一个 64 字节对齐的块里取指令**。用这一条解释你看到的现象。（提示：一个循环体如果正好跨在块边界上，每次迭代要取几次？）

验证你的解释——查一下取指单元一共重新访问了多少次 I-Cache：

```
grep -E 'fetch.cacheLines|icacheStallCycles' out/c_<名字>/stats.txt
```

把这两个数也填进表里。`icacheStallCycles` 占总周期的百分之几？5. 由此总结**两条**方法学规则：- 什么时候可以做”两次测量相减”，什么时候不可以？- 本课程的 `bench/Makefile` 为什么要强制加对齐参数？（打开 Makefile 看那段注释，但请先自己想）

> 这是本次实验的隐藏彩蛋，也是最值得写进报告的一段。同一份 C 代码，一个编译参数的差别，性能差 3 倍—— 而这 3 倍**和你要研究的任何设计决策都无关**。做体系结构实验，一半的功夫花在把这类干扰项摁住。

任务 3　四个 CPU 模型（30 分钟）

```
python3 framework/sweep.py --name lab1_cpu \
    --bench qnn --bench-args "8 < 学号后四位 >" \
    --vary cpu=atomic,timing,minor,o3 --jobs 4
```

```
python3 framework/report.py results/lab1_cpu.csv \
    insts cycles ipc host_seconds host_inst_rate l1d_miss_rate l1d_mshr_mpki
```

填表并作图（横轴仿真速率，纵轴 IPC，四个点标出模型名）：

模型	周期数	IPC	主机耗时	仿真速率 (指令/秒)	相对 o3 的 IPC 误差

要回答：1. `atomic` 和 `minor` 的 IPC 很接近，但主机耗时差 4 倍多。这说明 `atomic` 可以替代 `minor` 吗？（提示：看 L1D 失效率，再想想它们为什么” 碰巧” 接近）2. `timing` 的 IPC 比 `atomic` 低得多，可它俩都是” 一次一条指令”。差别从哪来？3. `o3` 的 IPC 是 `minor` 的 6 倍。**这 6 倍具体是被哪些机制变出来的？**（逐条说明：多发射贡献多少、乱序贡献多少、访存重叠贡献多少—— 现在只需给出定性推断，第 4 次实验会让你定量拆开）

任务 4　便宜的模型会不会骗你（30 分钟）【重要】核心任务

这是本次实验最重要的一步。我们要检验的不是” 模型准不准”，而是 **“模型会不会让我选错方案”**。

设计问题：L1 数据 Cache 选 16kB 还是 128kB？

```
python3 framework/sweep.py --name lab1_fid \
    --bench qnn --bench-args "8 < 学号后四位 >" \
    --vary cpu=timing,minor,o3 --vary l1d-size=16kB,128kB --jobs 6
python3 framework/report.py results/lab1_fid.csv cycles ipc l1d_miss_rate l1d_mshr_mpki
```

填表：

模型	16kB 周期数	128kB 周期数	该模型选谁	选对了吗
timing				
minor				
o3				

（以 o3 为” 参考答案”。真正的参考答案当然是真实芯片，但在本课程范围内 o3 是我们能拿到的最接近真实的模型。）

要回答：1. 三个模型给出了几种不同的结论？把每一种结论背后的机制说清楚。2. `timing` 认为大 Cache **更差**。它错在哪？（提示：`framework/cssys_se.py` 里的 `scaled_latency()`—— Cache 容量翻倍会让命中延迟 +1 周期。TimingSimpleCPU 会把这个代价放大成什么样？）3. `minor` 认为两者**几乎一样**。它错在哪？4. 由此，为后面六次实验写下你的模型选择，并说明理由。如果某次实验只关心指令数（不关心时序），你会不会换用更快的模型？

任务 5　工作负载要多大才够（20 分钟）

仿真很贵，负载越小越好；但太小的负载不能代表真实行为。找到那个平衡点：

```
for n in 2 4 8 16 32; do
  $GEM5_BIN --outdir=out/scale_$n framework/cssys_se.py \
      --cmd bench/bin/riscv/qnn --options "$n < 学号后四位 >" --cpu o3 &
done
wait
for n in 2 4 8 16 32; do
  echo -n "$n: "; python3 framework/stats.py out/scale_$n/stats.txt | \
      python3 -c "import json,sys; d=json.load(sys.stdin); print(d['insts'], round(d['ipc'],4), d['ho
done
```

画两张图：**IPC vs 样本数**、**主机耗时 vs 样本数**。

要回答：1. IPC 从第几个样本数开始基本不变了？变化幅度是多少？2. 主机耗时和样本数是什么关系？3. 如果一次设计空间扫描有 20 个配置点，你的机器有 8 个核，你选多大的样本数？把”扫描总耗时”算出来支持你的选择。4. 什么样的程序**不能**用”减小规模”这一招？（想想工作集和 Cache 容量的关系）

任务 6（选做）　换个负载验证结论

用 `pchase`（随机指针追逐，故意打败预取器）重做任务 4：

```
python3 framework/sweep.py --name lab1_fid_pchase \
    --bench pchase --bench-args "256 200000 < 学号后四位 >" \
    --vary cpu=timing,minor,o3 --vary l1d-size=16kB,128kB --jobs 6
```

要回答：换了负载以后，各模型的排序结论变了吗？如果变了，说明”模型的保真度”是负载相关的——这对你的模型选择意味着什么？

八、思考题（写进报告）

1. 有人说”模拟器误差 20%，所以模拟器没用”。用本次实验的数据反驳或支持这个观点。你的论证必须区分绝对保真和排序保真。
2. `hostInstRate`（每秒仿真多少条目标指令）大约是 3×10^5。假设你要仿真一个跑 1 秒的真实程序（按 2GHz、IPC=1 估算指令数），用 o3 模型需要多久？这个数字对”能研究什么样的问题”意味着什么？工业界用什么办法绕过它？（提示：抽样仿真 SimPoint、checkpoint、FPGA 加速）

3. 本次实验我们通过” 限制样本数” 来控制仿真成本。另一种常见做法是 `--maxinsts N`（跑够 N 条指令就停）。这两种做法在方法学上有什么区别？哪种更容易得出错误结论？

4. 我们的 ROI 只包含推理循环，不包含权重初始化。如果某个设计方案（比如一个大 Cache）需要” 预热” 才能发挥作用，这样划 ROI 会不会对它不公平？你会怎么改进？

九、提交要求

一份实验报告（PDF 或 Markdown），包含：

- 任务 1~5 的全部表格和图（图必须有坐标轴标签和单位）
- 每个” 要回答” 问题的回答，**结论必须指向你自己跑出来的数据**
- 一段不超过 300 字的**模型选择声明**：后面六次实验你用哪个模型、为什么、你预计它在什么情况下会误导你

同时提交 `results/` 下你生成的 CSV 文件。

> 评分看的是**分析质量**（40%），不是” 跑通了没有”（30%）。一张填满数字但没有解释的表格，得分低于一张只有三个点但把机制说清楚的图。

附录：常见问题

Q: `--outdir` 里没有 `stats.txt` 仿真崩了。看 `run.log` 或直接看终端输出。最常见的原因是 `--cmd` 路径写错，或者忘了给 `--options` 加引号导致参数被 shell 拆开。

Q: 第 0 段的指令数是 0 你跑的是 `_noroi` 版本，它没有 ROI 标记，整个文件只有一段。

Q: 两个同学的曲线拐点不在同一个位置正常，seed 不同。但**趋势**应该一致；如果趋势都不一样，其中一个人配置错了。

Q: 并行跑的时候机器卡死 `--jobs` 设得比物理核数还大了。gem5 单进程占内存约 1GB，注意内存也是约束。